Chương 2: Phân Tích Nghiệp Vụ & MVP Scoping

Khắc Phục Hội Chứng Nhồi Nhét Tính Năng và Chuẩn Hóa Đặc Tả Kỹ Thuật

Giáo Trình Kỹ Nghệ Vibecoding Chuyên Sâu 2026

Tháng 10 Năm 2026

Tóm tắt nội dung

Hội chứng thất bại phổ biến nhất của các dự án phần mềm do AI tạo ra không nằm ở trình độ viết mã, mà bắt nguồn từ sự thất bại trong khâu phân tích nghiệp vụ: người phát triển không biết tính năng nào là thực sự cần thiết, dẫn đến việc nhồi nhét tràn lan các chức năng phụ trợ, làm loãng trải nghiệm và khiến cơ sở mã phình to ngoài tầm kiểm soát. Chương 2 cung cấp phương pháp luận phân tích nghiệp vụ chuẩn mực dành riêng cho Vibecoding: áp dụng khung *Jobs-to-be-Done* (JTBD), sàng lọc tính năng theo thang đo *MoSCoW*, thực thi nghiêm ngặt *Quy tắc Cắt 50% tính năng*, và cô đọng yêu cầu thành tệp đặc tả kỹ thuật máy đọc được (`spec.md`) thông qua kỹ thuật phỏng vấn đối kháng Socratic.

---

1 Bản chất bài toán nghiệp vụ: Tại sao người dùng có xu hướng nhồi nhét tính năng?

Khi chi phí sinh mã nguồn giảm về gần bằng không nhờ trí tuệ nhân tạo, một cạm bẫy tâm lý mới xuất hiện: **Ảo tưởng về sự phong phú (Feature Bloat Illusion)**. Trong lập trình truyền thống, sự tốn kém về thời gian và công sức buộc kỹ sư phải đắn đo từng tính năng. Ngược lại, khi vibecoding, chỉ cần một câu prompt ngắn: *"Làm thêm cho tôi phần đăng nhập Google, dark mode, bảng xếp hạng, xuất file PDF và thông báo đẩy"*, AI sẽ lập tức sinh ra mã nguồn cho toàn bộ yêu cầu đó trong vài phút.

Tuy nhiên, hệ quả trực tiếp của việc nhồi nhét tính năng là:

1. **Phân mảnh luồng trải nghiệm (UX Dilution)**: Người dùng cuối bị choáng ngợp bởi hàng loạt nút bấm, menu đa cấp và bảng dữ liệu không có giá trị chuyển đổi cốt lõi.
2. **Bùng nổ trạng thái biên (Edge Case Explosion)**: Mỗi tính năng thêm vào làm tăng số lượng trạng thái logic theo cấp số nhân (2^N), dẫn đến xung đột dữ liệu và xuất hiện lỗi tiềm ẩn ở các module không ngờ tới.
3. **Cạn kiệt bộ nhớ chú ý của AI**: Khi cơ sở mã vượt quá 5.000 dòng mã với hàng chục module dở dang, mô hình AI mất khả năng ghi nhớ toàn vẹn cấu trúc dự án, bắt đầu sinh ra mã trùng lặp và phá vỡ logic cũ.

2 Khung lý thuyết Jobs-to-be-Done (JTBD) trong bối cảnh Vibecoding

Nguyên lý nền tảng của JTBD do Giáo sư Clayton Christensen (Đại học Harvard) khởi xướng: **"Khách hàng không đơn thuần mua một sản phẩm hay tính năng, họ 'thuê' sản phẩm đó để hoàn thành một bước tiến trong cuộc sống của họ."**

Trong Vibecoding, trước khi mở công cụ gõ lệnh, lập trình viên bắt buộc phải cấu trúc hóa bài toán theo công thức chuẩn:

> **Công thức JTBD chuẩn:**
> *Khi gặp hoàn cảnh [Bối cảnh & Nỗi đau cụ thể], tôi muốn [Hành động / Giải pháp trực tiếp], để tôi có thể đạt được [Kết quả chuyển đổi cốt lõi].*

2.1 Ví dụ phân tích: Ứng dụng Luyện thi TOEIC Reading

Xét một trường hợp điển hình mà người học hay gặp:

- **Cách tiếp cận sai (Tập trung tính năng râu ria)**: Yêu cầu AI làm app luyện thi có bảng xếp hạng bạn bè, shop đổi quà bằng điểm thưởng, chế độ thi thử 200 câu tính giờ, và diễn đàn thảo luận cộng đồng. Kết quả: App phức tạp, logic chấm điểm bị lỗi, giao diện lòe loẹt nhưng học sinh vẫn không đọc hiểu được bài thi.
- **Cách tiếp cận đúng (Theo JTBD)**: *"Khi gặp một câu hỏi Part 5 chứa nhiều từ mới và cấu trúc ngữ pháp phức tạp, tôi muốn câu hỏi đó được bóc tách từ vựng đơn giản hóa trước để làm chủ ngữ pháp, sau đó mới tích hợp từ vựng học thuật, để tôi có thể tự tin suy luận đáp án mà không bị hoảng loạn."*

Từ câu định vị JTBD chuẩn mực trên, sản phẩm chỉ cần **đúng một tính năng sát thủ (Killer Feature)**: *Bộ chuyển đổi câu hỏi phân tầng (Deconstructed Question Engine)*, mọi thứ khác (đăng nhập mạng xã hội, gamification) đều là rác thải nhận thức trong phiên bản đầu tiên.

3 Thang đo MoSCoW và Quy tắc Cắt 50% Tính Năng

Để ngăn chặn việc đưa các yêu cầu thừa vào dự án, toàn bộ ý tưởng ban đầu phải được đi qua bộ lọc **MoSCoW** và áp dụng nghiêm ngặt **Quy tắc Cắt 50% (The Cut-50% Rule)**.

Bảng 1: Ma trận phân loại tính năng MoSCoW dành cho dự án Vibecoding MVP

Cấp độ MoSCoW	Định nghĩa kỹ thuật	Ví dụ cho App Học tập	Hành vi trong Vibecoding
Must Have (Bắt buộc)	Nếu thiếu tính năng này, sản phẩm hoàn toàn vô dụng, không giải quyết được JTBD	Bóc tách câu hỏi thành 2 tầng: Ngữ pháp cô lập & Từ vựng cô lập	Khóa cứng trong `spec.md`, viết kiểm thử bao phủ 100%
Should Have (Nên có)	Quan trọng nhưng có thể giải quyết tạm thời bằng thao tác thủ công	Lưu lịch sử các câu đã làm vào bộ nhớ cục bộ (LocalStorage)	Chỉ xem xét sau khi Must Have đã chạy trơn tru
Could Have (Có thể có)	Hữu ích nhưng không ảnh hưởng trực tiếp đến giá trị sống còn	Chuyển đổi giao diện Sáng/Tối (Dark Mode), xuất PDF đề thi	Kiên quyết loại bỏ khỏi prompt phiên bản đầu
Won't Have (Tuyệt đối không)	Các tính năng gây xao nhãng hoặc phức tạp hóa kiến trúc quá sớm	Hệ thống nạp thẻ thanh toán, diễn đàn chat thời gian thực, bảng tin	Đưa vào danh sách cấm (*Non-Goals*) trong bản đặc tả

3.1 Quy tắc Cắt 50% tính năng (The Cut-50% Rule)

Quy tắc thực hành: Sau khi liệt kê toàn bộ danh sách tính năng dự kiến làm trong phiên bản thử nghiệm:

1. Gạch bỏ ngay lập tức 50% số tính năng kém quan trọng nhất (nhóm Could Have và Won't Have).
2. Khóa phạm vi chỉ xoay quanh: **1 Chân dung người dùng (Single Persona) + 1 Luồng công việc chính (Single Core Loop) + 1 Chỉ số đo lường giá trị duy nhất (Single North Star Metric)**.
3. Không bao giờ để AI tự ý bổ sung thêm bất kỳ màn hình hoặc nút bấm phụ nào ngoài danh mục Must Have đã duyệt.

4 Kỹ thuật Phỏng vấn Đối kháng Socratic (/grill-me)

Một sai lầm chí mạng khác của người vibecode là dùng câu lệnh một chiều: chỉ ra lệnh cho AI làm, trong khi AI vốn dĩ có xu hướng chiều lòng người dùng (*sycophancy*) và sẽ luôn đồng ý triển khai dù ý tưởng kiến trúc có vô lý hoặc chứa đầy mâu thuẫn nghiệp vụ.

Để khắc phục điều này, ta áp dụng kỹ thuật **Phỏng vấn Đối kháng Socratic** (hay lệnh điều phối `/grill-me`):

> **Khuôn mẫu lệnh phỏng vấn Socratic:**
> `Tôi muốn xây dựng tính năng [Tên ý tưởng]. Bạn hãy đóng vai một Giám đốc Sản phẩm khó tính và một Kỹ sư Trưởng hệ thống. TUYỆT ĐỐI CHƯA VIẾT CODE. Hãy đặt cho tôi 5 câu hỏi phản biện sắc bén nhất về: (1) Ranh giới nghiệp vụ thực tế, (2) Những trường hợp biên (edge cases) có thể làm vỡ logic dữ liệu, và (3) Tính năng nào trong ý tưởng này là râu ria thừa thãi cần cắt bỏ ngay.`

Quy trình đối thoại này ép người phát triển phải suy nghĩ thấu đáo về mô hình dữ liệu và các ranh giới thất bại trước khi bắt tay vào triển khai.

5 Chuẩn hóa tài liệu đặc tả máy đọc được (spec.md)

Sau khi phỏng vấn và lọc phạm vi, toàn bộ kết quả phải được kết tinh thành một tệp tài liệu duy nhất đặt tại gốc dự án mang tên `spec.md`. Đây là bản hợp đồng bất biến giữa con người và các tác tử AI.

Listing 1: Cấu trúc chuẩn của một tệp đặc tả kỹ thuật spec.md

```
# SPECIFICATION: DECONSTRUCTED TOEIC ENGINE (MVP v1.0)

## 1. CORE JTBD & VALUE PROPOSITION
- Persona: Nguoi hoc gap kho khan trong viec suy luan ngu phap khi bi can
    tro boi tu vung la.
- Core Loop: Nhan 1 cau hoi kho -> Tach thanh 2 tang bai tap rieng biet (
    Ngu phap / Tu vung) -> Gop lai lam bai hoan chinh.

## 2. MUST-HAVE SCOPE (KILLER FEATURE)
- Input: Doan van ban cau hoi goc gom 4 dap an A, B, C, D.
- Processor: Deconstruction Pipeline tach cau thanh 2 bien the.
- Output: Giao dien hien thi step-by-step 3 buoc ro rang.

## 3. NON-GOALS (TUYET DOI KHONG LAM TRONG V1)
- KHONG lam he thong dang nhap/dang ky (User Authentication).
- KHONG tich hop co so du lieu Cloud (Dung Local State / Mock JSON).
- KHONG lam Dark Mode, thong bao day hoac bang xep hang.
- KHONG ho tro xuat file PDF hoac in an.

## 4. DATA CONTRACT (TYPESCRIPT SCHEMA)
export interface QuestionStep {
  stepIndex: 1 | 2 | 3;
  type: 'ISOLATED_GRAMMAR' | 'ISOLATED_VOCAB' | 'SYNTHESIZED';
  instruction: string;
  stem: string;
  options: string[];
  correctAnswer: string;
  reasoning: string[];
}
```

6 Cây Quyết Định Định Hình Sản Phẩm & Ba Hướng Đi Cho Vibecoder

Sau khi phân tích JTBD và lọc danh sách Non-Goals, người học đứng trước ngã rẽ lựa chọn mô hình kiến trúc phù hợp. Tùy thuộc vào mục tiêu sản phẩm, giáo trình đề xuất 3 hướng đi thực chiến:

Bảng 2: Cây quyết định lựa chọn hướng đi sản phẩm sau bước phân tích nghiệp vụ

Hướng đi sản phẩm	Chiến lược phạm vi (Scope)	Kiến trúc kỹ thuật tối ưu	Chỉ số thành công (Metric)
Hướng 1: Micro-SaaS Đơn Tính Năng (Killer SaaS)	1 tính năng sát thủ duy nhất, không bắt đăng ký rườm rà, giải quyết tức thì	Next.js / Vite + Server Actions + Stripe / LemonSqueezy	Tỷ lệ chuyển đổi người dùng dùng thử thành công $> 35\%$
Hướng 2: Tiện Ích Cục Bộ (Local-First Tool)	Hoạt động 100% offline, không phụ thuộc đám mây, bảo mật dữ liệu tuyệt đối	Python / Tauri + SQLite FTS5 + PocketBase nhúng	Tốc độ khởi động < 1s, dung lượng bộ cài $<$ 15MB
Hướng 3: Tự Động Hóa & Bot Dòng Lệnh (CLI Automation)	Không cần UI, tập trung xử lý luồng công việc lặp đi lặp lại	Python CLI (argparse/click) + Linux Bash / Windows PowerShell	Tiết kiệm thời gian thao tác thủ công từ hàng giờ về vài giây

6.1 Kịch bản thực hành hỗ trợ: socratic_spec_builder.py

Dự án đi kèm công cụ dòng lệnh `socratic_spec_builder.py` cho phép người học tự động hóa việc sinh tệp `spec.md` chuẩn mực chỉ bằng cách nhập 5 tham số đầu vào của JTBD, tự động thiết lập danh sách Non-Goals bị cấm và tích hợp các ràng buộc an ninh chống AI Slop.

7 Tổng kết chương và bài tập thực hành

7.1 Tóm tắt cốt lõi

- Thất bại của sản phẩm AI phần lớn là do thất bại về phạm vi nghiệp vụ, không phải do cú pháp ngôn ngữ lập trình.
- Khung JTBD giúp định vị chính xác nỗi đau cốt lõi thay vì chạy theo các tính năng hào nhoáng bên ngoài.
- Áp dụng triệt để Quy tắc Cắt 50% và bảng MoSCoW để bảo vệ bộ nhớ ngữ cảnh của AI và sự tập trung của sản phẩm.
- Luôn khóa cứng nghiệp vụ trong tệp `spec.md` với danh sách *Non-Goals* minh bạch trước khi viết dòng code đầu tiên.

7.2 Bài tập thực hành

1. **Thực hành JTBD**: Chọn một ý tưởng ứng dụng bạn đang ấp ủ. Hãy viết lại mục tiêu của ứng dụng theo đúng công thức JTBD 3 vế chuẩn mực.
2. **Cắt gọt phạm vi**: Liệt kê 10 tính năng bạn từng nghĩ sẽ đưa vào app đó. Thực hiện cắt bỏ đúng 5 tính năng và giải thích lý do tại sao loại bỏ chúng mà sản phẩm vẫn giữ nguyên giá trị cốt lõi.
3. **Khởi tạo spec.md**: Soạn thảo một tệp `spec.md` hoàn chỉnh theo cấu trúc chuẩn ở Mục 5 cho bài toán bạn đã chọn.